

Quiz4 (A) :Introduction to Programming

Name:

Date: 12/12/2025

ERP

Note: All questions from 1- 7 are 2 marks each.

1. Question 1:

```
struct Node {
    int data;
    Node* next;
};

int main() {
    Node* head = new Node{10, nullptr};
    head->next = new Node{20, nullptr};
    head->next->next = new Node{30, nullptr};

    Node* ptr = head;
    while (ptr->next != nullptr) {
        ptr = ptr->next;
    }
    cout << ptr->data << endl;
    return 0; }
```

Options:

- A. 10
- B. 20
- C. 30
- D. The code results in a segmentation fault.

2. Question 2:

```
struct Node {
    int data;
    Node* next;
};
```

```

int length(Node* head) {
    int count = 0;
    while (head != nullptr) {
        count++;
        head = head->next;
    } return count; }
int main() {
    Node* head = nullptr;
    cout << length(head) << endl;
    return 0;}

```

Options:

- A. 0
- B. 1
- C. Undefined behavior
- D. The program will crash

3. Question 3:

```

struct Node {
    int data;
    Node* next;};

void insertFront(Node*& head, int val) {
    Node* newNode = new Node{val, head};
    head = newNode; }

int main() {
    Node* head = new Node{10, nullptr};
    insertFront(head, 5);
    cout << head->data << endl;
    return 0;}

```

Options:

- A. 5
- B. 10
- C. It depends on the system
- D. Code won't compile

4. Question 4:

```
struct Node {
    int data;
    Node* next;};

Node* findNode(Node* head, int key) {
    while (head != nullptr && head->data != key) {
        head = head->next;
    } return head;}

int main() {
    Node* head = new Node{1, nullptr};
    head->next = new Node{2, nullptr};
    head->next->next = new Node{3, nullptr};

    Node* result = findNode(head, 2);
    cout << (result == nullptr ? -1 : result->data) << endl;
    return 0;}
```

Options:

- A. -1
- B. 1
- C. 2
- D. 3

Note: All tracing questions from 5-7 are 3 marks each

5. Question

```
struct Node {
    int data;
    Node* next; };
void deleteAfter(Node* node) {
    if (node == nullptr || node->next == nullptr)
        return;
    Node* temp = node->next;
    node->next = temp->next;
    delete temp; }
```

```

int main() {
    Node* head = new Node{100, nullptr};
    head->next = new Node{200, nullptr};
    head->next->next = new Node{300, nullptr};

    deleteAfter(head);
    Node* ptr = head;
    while (ptr != nullptr) {
        cout << ptr->data << " ";
        ptr = ptr->next;
    }
    cout << endl;
    return 0;
}

```

6. Question

```

struct Node {
    int data;
    Node* next;
};

```

```

Node* reverseList(Node* head) {
    Node* prev = nullptr;
    Node* curr = head;
    while (curr != nullptr) {
        Node* nextNode = curr->next;
        curr->next = prev;
        prev = curr;
        curr = nextNode;
    }
    return prev;
}

```

```

int main() {
    Node* head = new Node{10, nullptr};
    head->next = new Node{20, nullptr};
    head->next->next = new Node{30, nullptr};

    head = reverseList(head);
    cout << head->data << endl;
    return 0;
}

```

Question : After calling reverseList, what is head->data?

7. **Question :**

```
struct Node {
    int data;
    Node* next; };
void printMiddle(Node* head) {
    Node *slow = head, *fast = head;
    while (fast != nullptr && fast->next != nullptr) {
        slow = slow->next;
        fast = fast->next->next;
    }
    if (slow != nullptr)
        cout << slow->data << endl;
    else
        cout << "Empty list" << endl; }

int main() {
    Node* head = new Node{5, nullptr};
    head->next = new Node{10, nullptr};
    head->next->next = new Node{15, nullptr};
    printMiddle(head);
    return 0; }
```

Question: What is the output of printMiddle(head)?

Q8. The code given aims to insert a node in the middle of a linked list. However, the function does not work as intended. **Identify the bug and fix it. [5]**

```
1. struct Node {
2.     int data;
3.     Node* next;
4. };
5. Node* insertMiddle(Node* head, int value) {
6.     if (head == nullptr) {
7.         return new Node{value, nullptr};
8.     }
9.     Node* slow = head;
10.    Node* fast = head;
11.    while (fast != nullptr && fast->next != nullptr) {
12.        slow = slow->next;
13.        fast = fast->next->next;
14.    }
15.    Node* newNode = new Node{value, slow->next};
16.    slow->next = newNode;
17.    return head;
18. }
```